1. What is the difference between JDK, JRE, and JVM?

JVM is the runtime engine that executes bytecode.

JRE includes the JVM and core libraries needed to run Java programs.

JDK includes the JRE plus development tools such as the compiler and debugger.

2. What are the main differences between primitive types and reference types?

Primitive types store actual values and are not objects.

Reference types store addresses to objects on the heap.

Reference types support methods and object behaviors.

3. Is Java pass-by-value or pass-by-reference? Explain.

Java is always pass-by-value.

Primitive values are copied directly.

Object references are copied, meaning the method receives a copy of the reference.

4. Why are Strings immutable in Java?

For security purposes.

For performance and hashcode caching.

For thread-safety.

To support string pooling.

5. What is the String Constant Pool and how does it work?

It is a special heap area storing string literals.

Identical literals refer to the same pooled object.

Using 'new' creates a separate object unless interned.

6. What is the purpose of the final keyword in Java?

Final variables cannot change after assignment.

Final methods cannot be overridden.

Final classes cannot be subclassed.

7. What does the static keyword mean for variables or methods?

Static members belong to the class rather than any instance.

Static methods and variables can be accessed without creating objects.

8. What is a static block and when does it run?

A static block initializes static members.

It runs once when the class is loaded.

9. Can a static method access non-static variables? Why or why not?

A static method cannot directly access non-static variables.

Static methods lack a 'this' reference.

They must use an object instance to access instance variables.

10. Describe the JVM loading order.

Static fields are initialized first.

Static blocks run next.

Instance fields initialize when objects are created.

Constructors run last.

11. Difference between a static variable and a public static final constant?

Static variables can be modified.

Public static final constants cannot be changed.

Final constants are often inlined by the compiler.

12. Why are immutable objects thread-safe by design?

Their state cannot change after creation.

There are no race conditions involving internal state.

No synchronization is required for access.

13. What problem does making a class final solve?

It prevents subclasses from overriding methods.

It protects immutability.

It prevents breaking class behavior through inheritance.